

TECHNICAL REPORT

Adaptive Machine Learning for Dynamic Radiolocation

Benjamin Philipenko

Department of Electrical and Computer Engineering, University of Victoria

Fall 2024

Part 1 — Interim Report (November 2024): problem background, Parzen window estimator vs. neural network localization, comparative testing

Part 2 — Final Report (December 2024): tracking a moving target with general and adaptive Kalman filters, process-noise optimization, neural network comparison

Appendix: helper functions and references

Parzen Window Estimation for WLAN Terminal Localization

Submitted November 2024

Problem Background

Indoor radiolocation systems face a significant challenge in environments that frequently change, where factors such as the repositioning of base stations, movement of large objects affecting radio propagation, body shadowing, and high foot traffic can significantly alter signal behavior [1]. These conditions are common in urban settings and real-world indoor environments, making accurate location estimation difficult. This project aims to develop adaptive machine learning techniques capable of maintaining high accuracy in radiolocation systems, even in dynamic environments where significant changes occur.

Previous work addressing these challenges faced in indoor radiolocation includes a UVIC PhD project involving an autonomous drone that periodically updates the survey data used to estimate the location of mobile devices [2]. This autonomous robot successfully traced a predefined path to collect wireless packets and produced a database of RSSI and CSI fingerprints alongside a 2D LIDAR map of the floor. While effective, this solution has significant limitations: data collection is time-consuming and can only occur infrequently, often restricted to once a night or, more realistically, once a month. In dynamic in-building environments, this can lead to outdated or inaccurate survey data, compromising the precision of location estimates as environmental changes occur daily.

The goal of this project is to develop a neural network that improves indoor localization in Wireless Local Area Networks (WLANs) by utilizing received signal strength (RSS) data from multiple wireless access points (WAPs), a solution that has been proposed for various mobile computing applications [1]. This WLAN terminal location technique offers several advantages: it does not require additional measurement hardware, it operates without the need for strict synchronization between base stations, and it can be implemented across various cellular network configurations with minimal adjustments [1]. Rather than creating a new WLAN location estimation algorithm, this research focuses on designing an adaptive machine learning method capable of adjusting to environmental changes. By doing so, it eliminates the need for constant re-collection of survey data whenever the environment or base station locations are altered. A single calibration set will serve as the reference, enabling accurate and efficient location-aware computing inside buildings without the need for repeated data collection.

Simulation Environment and Dataset Configuration

A simulation environment must be defined with randomly generated data points to use for the initial development and testing of the project. The initial simulation environment consists of a 100m x 100m grid, with no obstacles. Four base stations are located at (25, 25), (25, 75), (75, 25), and (75, 75). Random sample data points will be generated, which will include three parameters: an x-coordinate, a y-coordinate, and an

RSS measurement vector corresponding to its position relative to each base station in the simulation environment. To account for the non-linear distribution of signal propagation and attenuation [3], each RSS measurement vector has a path loss vector, which represents the measured path loss value in decibels corresponding to the location of a given survey data point. The measured path loss values are modeled using the path loss formula for isotropic antennas expressed in terms of decibels:

$$PL = g(\theta) + V$$

$$g(\theta) = 20 \log_{10} \left(\frac{4\pi df}{c} \right)$$

$$V \sim N(0, \sigma^2)$$

where $g(\theta)$ is a vector function which gives the true median path loss values for locations θ [3], d is the distance between the nearest base station transmitter and mobile receiver, f is the signal frequency, and c is the speed of light [4]. V is a zero-mean random error vector that is independent of the location θ . It is crucial to account for the radio path loss within the model to simulate the real-world attenuation of radio signals, providing a realistic RSS estimate at any given location. The randomly generated values within the simulation are treated as 100% accurate. These values represent a real-life dataset, as if collected from a mobile device measuring RSS from multiple WAPs at a mobile terminal.

```
% Simulation environment parameters
grid_size = 100;           % Size of grid (100m x 100m)
base_stations = [25, 25;   % Base station location 1
                 25, 75;   % Base station location 2
                 75, 25;   % Base station location 3
                 75, 75]; % Base station location 4

% Location Measurements (2xN)
Y_train = grid_size * rand(800, 2); % Training Data Set
Y_valid = grid_size * rand(100, 2); % Validation Data Set
Y_eval = grid_size * rand(100, 2); % Evaluation Data Set

% Path loss model parameters
A = -30; % RSS at reference distance of 0m (perfect signal connection), in dBm
f = 2.4e9; % Frequency in Hz (2.4 GHz for WLAN/WiFi)
c = 3e8; % Speed of light in m/s

% RSS Measurements (4xN)
X_train = zeros(800, 4); % Training Data Set
X_valid = zeros(100, 4); % Validation Data Set
X_eval = zeros(100, 4); % Evaluation Data Set
for ii = 1 : 4

    % Compute Euclidean distance to the base station
    distance_to_bs_train = sqrt( sum( abs( base_stations( ii , :) - Y_train ).^2 , 2 ) );
    distance_to_bs_valid = sqrt( sum( abs( base_stations( ii , :) - Y_valid ).^2 , 2 ) );
    distance_to_bs_eval = sqrt( sum( abs( base_stations( ii , :) - Y_eval ).^2 , 2 ) );

    % Apply path loss model (gaussian noise with variance = 1)
    X_train(:, ii) = A - 20 * log10(4 * pi * distance_to_bs_train * f / c) + (1 * rand);
    X_valid(:, ii) = A - 20 * log10(4 * pi * distance_to_bs_valid * f / c) + (1 * rand);
    X_eval(:, ii) = A - 20 * log10(4 * pi * distance_to_bs_eval * f / c) + (1 * rand);
end
```

end

WLAN Terminal Localization Algorithm

Parzen Window Estimator Method - Validation

The aim of this section is to carry out a cross-validation procedure, as described in [1], [5], to determine the optimal kernel width for use in a Parzen Window Estimator. This estimator, detailed in [1], [6], is employed as an approximation to the Minimum Mean Square Error (MMSE) estimator for predicting location coordinates based on RSS measurements. The kernel width is a crucial parameter in Parzen Windows, directly impacting the accuracy of the location estimates derived from RSS data.

To find the optimal kernel width, a range of kernel widths, selected from the recommended search spaces in [5], are systematically tested to identify the one that minimizes the MSE in the location estimates. Each kernel width within this range is assessed using a Leave-One-Out Cross Validation technique [7]. In each iteration, a specific survey point is omitted from the dataset, treating it as a validation point. The remaining data points serve as the training set from which weights for the Parzen Window Estimator are calculated using a Gaussian kernel.

For each left-out point, the location estimate is calculated as a weighted sum of the coordinates of the training points, where weights are based on the similarity of RSS values between the training and validation points. The squared error between the estimated location and the actual location of the omitted point is then calculated and accumulated across all survey points for the current kernel width to obtain the cross-validation MSE. For a sufficiently large survey set, the optimal kernel width is determined by finding the one that produces the minimum cross-validation MSE, indicating the best fit for the location estimation model [5].

```
% Define range of kernel widths to test
h_vector = 0.01 : 0.001 : 2;

% Initialize vector to store cross-validation MSE values for each kernel width
MSE_valid = zeros( size( h_vector ) );

% Perform Leave-One-Out Cross Validation for each kernel width
for hh = 1 : length( h_vector )

    % Test current kernel width
    h_value = h_vector( hh );

    % Estimate the weights using Gaussian kernel based on RSS similarity
    weights_valid = gaussian_kernel(X_valid, X_train, h_value);

    % Compute estimated locations
    Y_est = zeros( length(X_valid) , 2 );
    for nn = 1 : length( X_valid )

        % Estimate the location using weighted sum of training locations
        Y_est( nn , : ) = sum( weights_valid( nn , : ) .* Y_train.' , 2 );
```

```

end

% Compute cross-validation MSE
MSE_valid( hh ) = mean( sum( abs( Y_est - Y_valid ).^2 , 2 ) );

end

% Find the kernel width which gave the best MSE performance
[ ~ , h_best_idx ] = min( MSE_valid );
h_best = h_vector( h_best_idx );
fprintf('Optimal kernel width: %.6f\n', h_best);
plot( h_vector , MSE_valid , '-' , h_best , MSE_valid( h_best_idx ) , '*' );
title('Mean Squared Error (Validation)');
xlabel('Kernel Width (h)');

```

Optimal kernel width: 0.702000

Parzen Window Method - Evaluation MSE: 8.812738

Parzen Window Estimator Method - Evaluation

After obtaining the optimal kernel width during the Cross Validation process, all components required to implement the PWE method for performing localization have been obtained. This optimal kernel width is used to evaluate the Parzen Window Estimator on a separate evaluation set, producing an overall evaluation MSE that reflects the method's accuracy.

```

% Estimate the weights using Gaussian kernel based on RSS similarity
weights_eval = gaussian_kernel(X_eval, X_train, h_best);

% Compute estimated locations
Y_est = zeros( length(X_eval) , 2 );
for nn = 1 : length( X_eval )

    % Estimate the location using weighted sum of training locations
    Y_est( nn , : ) = sum( weights_eval( nn , : ) .* Y_train.' , 2 );

end

% Compute cross-validation MSE
MSE_eval = mean( sum( abs( Y_est - Y_eval ).^2 , 2 ) );
fprintf('Parzen Window Method - Evaluation MSE: %.6f\n', MSE_eval);

```

Evaluation MSE using Neural Network Localization Method: 24.50513278

Neural Network Method

The neural network method is constructed using a feedforward network with one hidden layer of 10 nodes. The network is trained on the training dataset using the measured RSS values as inputs and the location coordinates as outputs. After training, the network is tested on the evaluation dataset, with location predictions generated for each RSS measurement. The MSE between the predicted and actual locations in the evaluation dataset is computed as the performance metric, providing an alternative measure of accuracy to compare against the Parzen Window Estimator.

```

% Construct a feedforward network with one hidden layer of size 10.
net = feedforwardnet(10);

% Train the network
[net, tr] = train(net, X_train', Y_train');
% Test the network
Y = net(X_eval');

% Calculate performance
perf = perform(net, Y, Y_eval');

% Display the evaluation MSE
fprintf('Evaluation MSE using Neural Network Localization Method: %.8f\n', perf);

```

Comparative Testing for Different System Parameters

The aim of this section is to validate the effectiveness of the Parzen Window Estimator method for localization by comparing its MSE with that of a Neural Network model across different simulation parameters. The primary goal is to understand how the accuracy of both models changes with varying levels of signal variance and sample size.

Accuracy vs. Variance

In this test, the variance of the RSS vector was incrementally adjusted to simulate varying levels of signal attenuation. After 100 epochs of localization processing for each variance, the average MSE was calculated for both the PWE and NN models at each level of RSS vector variance. The results are as follows:

Variance (σ^2)	Parzen Window MSE	Neural Network MSE
0	4.2858	1.1234
1	10.1707	17.3771
2	25.8833	61.9988
3	52.0611	142.6524
4	90.7186	265.4620

From the results, it can be observed that both the PWE and NN models experience a significant increase in MSE as the variance of the RSS vector increases, indicating that both models struggle more as signal noise and attenuation become more prevalent. When the variance is zero, the NN model outperforms the PWE model, achieving a significantly lower MSE. However, as the variance increases, the MSE for both models increases, with the NN model showing an exponential increase. For example, at a variance of 4, the PWE model's MSE is 90.7186, while the NN model's MSE reaches 265.4620. This suggests that the PWE model is more stable under varying levels of signal attenuation, whereas the NN model becomes less accurate as the noise in the data increases, possibly due to overfitting or difficulty generalizing to noisy data.

The relative stability of PWE's performance as compared to the NN model suggests that PWE may be better suited for environments where signal attenuation and noise are higher, offering more reliable performance

under such conditions.

Accuracy vs. Sample Size

In this test, the sample size used in the validation and evaluation processes are incrementally adjusted to determine its impact on the accuracy of the models. After 10 epochs of localization processing for each sample size at a fixed variance of 1, the average MSE is calculated for both models. The results are as follows:

Sample Size ($\sigma^2 = 1$)	Parzen Window MSE	Neural Network MSE
1000	11.7223	18.0489
2000	6.7644	23.5574
3000	7.6137	29.2694
4000	6.1826	23.2884
5000	7.2167	27.2828
6000	8.3761	39.2940
7000	4.2103	14.2485
8000	6.8211	25.9808
9000	5.9845	20.7047
10000	6.6656	19.4780

At a variance of 1, the results show that the PWE method consistently outperforms the NN model, with lower MSE values across all sample sizes. While the MSE for both models generally decreases as the sample size increases, the PWE model experiences a more significant improvement with larger datasets. For example, at a sample size of 1000, the PWE model's MSE is 11.7223, while the NN model's MSE is 18.0489. As the sample size increases to 10,000, the PWE model's MSE reduces to 6.6656, while the NN model's MSE only slightly decreases to 19.4780. The NN model's MSE appears more erratic, with no clear trend of improvement. This pattern indicates that the PWE model scales more effectively with larger datasets, benefiting more from the increased data compared to the NN model.

To determine if a more distinct trend or pattern would emerge with higher variance, the same test was conducted again with a fixed variance of 2. After 10 epochs of localization processing for sample sizes of 1000 and 10,000, the average MSE was calculated for both models. The test was only run for these sample sizes due to time and computational resource constraints. The results are as follows:

Sample Size ($\sigma^2 = 2$)	Parzen Window MSE	Neural Network MSE
1000	31.7682	71.4876
10000	17.8135	87.5215

At a variance of 2, the pattern observed is similar, although the MSE values are higher for both models. For a sample size of 1000, the PWE model's MSE is 31.7682, while the NN model's MSE is 71.4876. With a larger sample size of 10,000, the PWE model's MSE decreases to 17.8135, while the NN model's MSE is 87.5215.

These results further reinforce the PWE model's superior performance, particularly in handling increased signal noise and larger datasets, where its MSE remains significantly lower than the NN model's.

The results of this comparative testing demonstrate that the PWE method generally outperforms the NN model in terms of localization accuracy, both in terms of variance in the RSS vector and sample size. The PWE method shows robustness in environments with higher signal noise, maintaining lower MSE values across various conditions. Additionally, the PWE method benefits more from larger sample sizes compared to the NN model, suggesting that it may be a more efficient approach for localization tasks, especially when dealing with high levels of noise and large data sets. These findings validate the PWE method as a viable and efficient localization technique for signal processing applications.

Kalman Filter Tracking for Dynamic Radiolocation

Submitted December 2024

Simulation Environment and Dataset Configuration

```
% Simulation environment parameters
grid_size = 100;           % Size of grid (100m x 100m)
sigma_noise = 2;           % Noise variance level
base_stations = [25, 25;   % Base station location 1
                 25, 75;   % Base station location 2
                 75, 25;   % Base station location 3
                 75, 75]; % Base station location 4

% Set size parameters
N_train = 8000;
N_valid = 1000;
N_eval  = 10000;

% Location Measurements (2xN)
Y_train = grid_size * rand( N_train , 2); % Training Data Set
Y_valid = grid_size * rand( N_valid , 2); % Validation Data Set
Y_eval  = grid_size * rand( N_eval  , 2); % Evaluation Data Set

% RSS Measurements (4xN)
X_train = zeros( N_train , 4); % Training Data Set
X_valid = zeros( N_valid , 4); % Validation Data Set
X_eval  = zeros( N_eval  , 4); % Evaluation Data Set

% Path loss model parameters
L0 = -30; % RSS at reference distance of 0m (perfect signal connection), in dBm
alpha = 2; % Path loss exponent
f = 2.4e9; % Frequency in Hz (2.4 GHz for WLAN/WiFi)
c = 3e8; % Speed of light in m/s

for ii = 1 : 4

    % Compute Euclidean distance to the base station
    distance_to_bs_train = sqrt( sum( abs( base_stations( ii , :) - Y_train ).^2 , 2 ) );
    distance_to_bs_valid = sqrt( sum( abs( base_stations( ii , :) - Y_valid ).^2 , 2 ) );
    distance_to_bs_eval  = sqrt( sum( abs( base_stations( ii , :) - Y_eval ).^2 , 2 ) );

    % Apply log-distance path loss model (gaussian noise with variance = sigma_noise^2)
    X_train(:, ii) = L0 - 10 * alpha * log10(4 * pi * distance_to_bs_train * f / c) + (sigma_noise * randn( N_train , 1 ) );
    X_valid(:, ii) = L0 - 10 * alpha * log10(4 * pi * distance_to_bs_valid * f / c) + (sigma_noise * randn( N_valid , 1 ) );
    X_eval(:, ii) = L0 - 10 * alpha * log10(4 * pi * distance_to_bs_eval * f / c) + (sigma_noise * randn( N_eval , 1 ) );

end
```

Parzen Window Estimator Method - Validation

```
% Define range of kernel widths to test
h_vector = 0.1 : 0.1 : 4;

% Initialize vectors to store cross-validation MSE values for each kernel width
MSE_loc_valid = zeros(size(h_vector));
MSE_rss_valid = zeros(size(h_vector));

% Perform Leave-One-Out Cross Validation for each kernel width
for hh = 1 : length(h_vector)

    % Test current kernel width
    h_value = h_vector(hh);

    % Estimate the weights using Gaussian kernel based on RSS similarity
    weights_loc_valid = gaussian_kernel(X_valid, X_train, h_value);

    % NOTE: Y data sets (locations) passed in as arguments for calculating weights to be used in rss
    estimation algorithm
    weights_rss_valid = gaussian_kernel(Y_valid, Y_train, h_value);

    % Compute estimated locations and RSS
    Y_loc_est = zeros( N_valid , 2); % Estimated locations
    X_rss_est = zeros( N_valid , 4); % Estimated RSS for each base station

    for nn = 1 : N_valid
        % Estimate the location using weighted sum of training locations
        Y_loc_est(nn, :) = sum(weights_loc_valid(nn, :) .* Y_train.', 2);

        % Estimate the RSS for all four base stations
        for ii = 1 : 4
            % Compute weighted sum of RSS values for each base station
            X_rss_est(nn, ii) = sum(weights_rss_valid(nn, :) .* X_train(:, ii).');
        end
    end

    % assert(size(Y_loc_est, 2) == 2, 'Y_loc_est must have 2 columns');

    % assert(size(X_rss_est, 2) == 4, 'X_rss_est must have 4 columns');

    % Compute cross-validation MSE for locations
    MSE_loc_valid(hh) = mean( sum( abs( Y_loc_est - Y_valid ).^2, 2 ) );

    % Compute cross-validation MSE for RSS
    MSE_rss_valid(hh) = mean( sum( abs( X_rss_est - X_valid ).^2, 2 ) );
end

% Find the kernel width which gave the best MSE performance for location
[~, h_best_loc_idx] = min(MSE_loc_valid);
h_x = h_vector(h_best_loc_idx);

% Find the kernel width which gave the best MSE performance for RSS
[~, h_best_rss_idx] = min(MSE_rss_valid);
h_y = h_vector(h_best_rss_idx);

% Display optimal kernel widths
fprintf('Optimal kernel width for location (hx): %.2f\n', h_x);
```

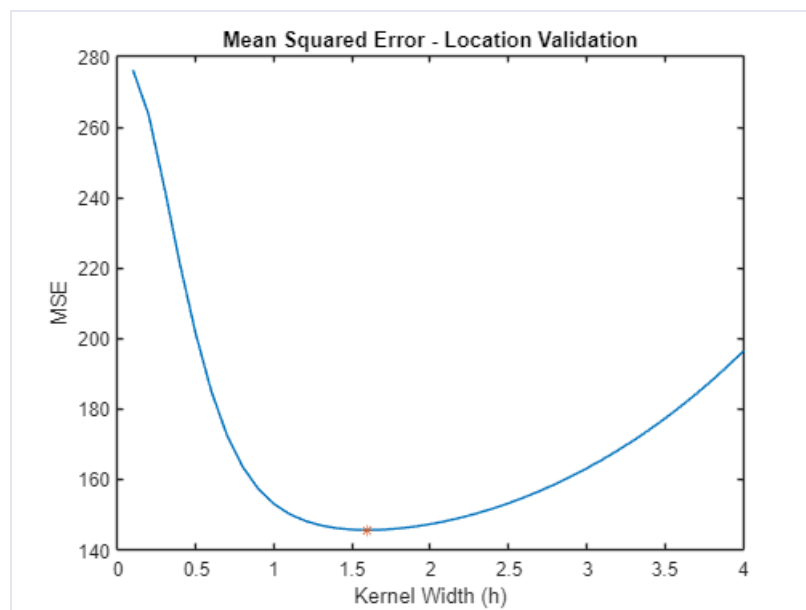
```

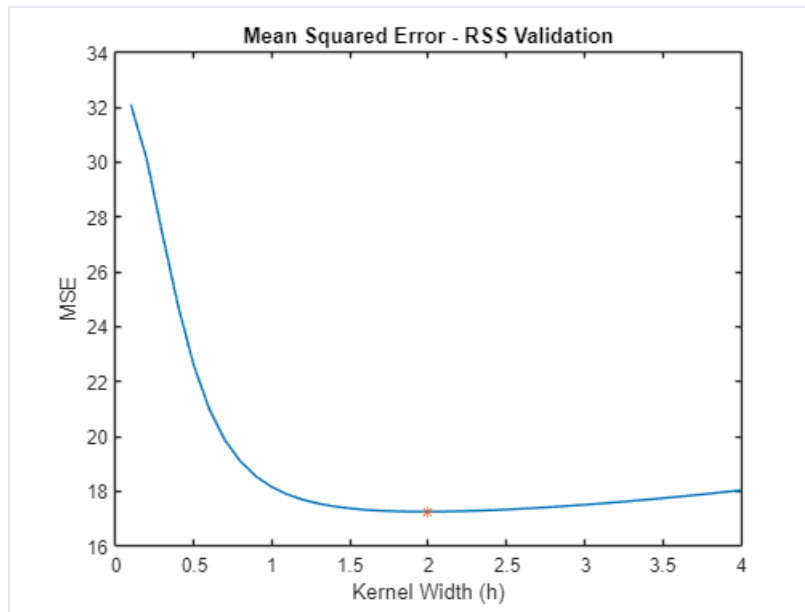
fprintf('Optimal kernel width for RSS (hy): %.2f\n', h_y);
% Plot MSE for location
figure;
plot(h_vector, MSE_loc_valid, '-', h_x, MSE_loc_valid(h_best_loc_idx), '*');
title('Mean Squared Error - Location Validation');
xlabel('Kernel Width (h)');
ylabel('MSE');
% Plot MSE for RSS
figure;
plot(h_vector, MSE_rss_valid, '-', h_y, MSE_rss_valid(h_best_rss_idx), '*');
title('Mean Squared Error - RSS Validation');
xlabel('Kernel Width (h)');
ylabel('MSE');

```

Optimal kernel width for location (hx): 1.60

Optimal kernel width for RSS (hy): 2.00





Parzen Window Estimator Method - Evaluation

```
% Estimate weights using Gaussian kernel
weights_loc_eval = gaussian_kernel( X_eval, X_train, h_x ); % RSS kernel weights
weights_rss_eval = gaussian_kernel( Y_eval, Y_train, h_y ); % Location kernel weights

% Initialize variance matrix
variance_matrices = zeros( 2 , 2 , N_eval ); % Location variance estimation

% Compute estimated locations
Y_loc_est = zeros( N_eval , 2 );
X_rss_est = zeros( N_eval , 4 );

% Variance term constants
cov_ky = (h_y^2) * eye(2); % Covariance of kernel (h_y^2 * I)

for nn = 1 : N_eval

    % Estimate the location using weighted sum of training locations
    Y_loc_est( nn , : ) = sum( weights_loc_eval( nn , : ) .* Y_train.' , 2 );

    % Initialize variance matrix
    expectation_yyT = zeros( 2 , 2 );

    % Compute weighted expectation
    for kk = 1 : N_train

        yk = Y_train( kk , : ).'; % Training location
        kernel_weight = weights_loc_eval( nn , kk );
        expectation_yyT = expectation_yyT + kernel_weight * ( yk * yk.' + cov_ky );

    end

    % Store variance matrix
    variance_matrices( : , : , nn ) = expectation_yyT - Y_loc_est( nn , : )' * Y_loc_est( nn , : );
```

```

% Estimate the RSS for all four base stations
for ii = 1 : 4
    % Compute weighted sum of RSS values for each base station
    X_rss_est( nn , ii ) = sum( weights_rss_eval( nn , : ) .* X_train( : , ii ).' );
end

end

% Compute cross-validation MSE
MSE_loc_eval = mean( sum( abs( Y_loc_est - Y_eval ).^2 , 2 ) );
fprintf('WLAN Localization - Evaluation MSE for sigma_noise = %.2f: %.4f\n', sigma_noise , MSE_loc_eval);
MSE_rss_eval = mean( sum( abs( X_rss_est - X_eval ).^2 , 2 ) );
fprintf('RSS Estimation - Evaluation MSE for sigma_noise = %.2f: %.4f\n', sigma_noise , MSE_rss_eval);
% Calculate the mean variance matrix
mean_variance_matrix = mean(variance_matrices, 3);
disp(mean_variance_matrix);

```

WLAN Localization - Evaluation MSE for sigma_noise = 2.00: 135.6275

RSS Estimation - Evaluation MSE for sigma_noise = 2.00: 16.6572

```

101.0401    0.4872
    0.4872   104.5327

```

Kalman Filter Implementation - General vs Non-General Variance Estimation

```

% Initialize Random Waypoint Motion Model
timeStep = 1; % Poll position every second
simulationTime = 120; % Total simulation time
true_position = grid_size * rand(1, 2); % Initial random starting position
stopPoint = grid_size * rand(1, 2); % Initial random stop point
speed = 1 + 2 * rand;

% State-transition matrix (location and velocity)
F = [1 0 timeStep 0;
     0 1 0 timeStep;
     0 0 1 0;
     0 0 0 1];

% Observation matrix
H = [1 0 0 0; % Only position is measured, so H matrix maps state to measurement
     0 1 0 0];

% Covariance of process noise
Q = [0.4 0 0 0; % Tuning values recommended by Dr. McGuire
     0 0.1 0 0;
     0 0 0.5 0;
     0 0 0 0.5];

% Covariance of measurement noise
R = mean_variance_matrix; % This value is the mean variance estimation calculated previously in evaluation
process for PWE

% Initial State Estimate (position and velocity)
x_k = [true_position, 0, 0]'; % General Variance Estimation

```

```

x_k_dynamic = x_k; % Non-General Variance Estimation

% Initial error covariance
P_k = [mean_variance_matrix, zeros(2, 2);
       zeros(2, 2), diag([10, 10])]; % General Variance Estimation
P_k_dynamic = P_k; % Non-General Variance Estimation

% Initialize matrices to store the real and predicted paths for plotting later
real_path = zeros( simulationTime , 2 );
pwe_estimated_path = zeros( simulationTime , 2 );
kalman_predicted_path = zeros( simulationTime , 2 );
dynamic_kalman_predicted_path = zeros( simulationTime , 2 );

% Main simulation loop
for t = 1 : timeStep : simulationTime

    % Save current real position
    real_path(t, :) = true_position;

    % Simulate RSS measurements
    rss_measurement = zeros(1, 4);
    for ii = 1 : 4
        distance_to_bs = norm( true_position - base_stations( ii , : ) ); % Distance to base station
        rss_measurement( ii ) = L0 - 10 * alpha * log10(4 * pi * distance_to_bs * f / c) + ( sigma_noise *
randn );
    end

    % Estimate position using PWE
    weights_loc = gaussian_kernel( rss_measurement , X_train , h_x );
    pwe_estimated_position = sum( weights_loc .* Y_train.' , 2 ).'; % PWE-estimated location

    % Save the PWE estimated position
    pwe_estimated_path( t , : ) = pwe_estimated_position;

    % Kalman Filter Implementation - General Variance Estimation
    % Predict the next state
    x_k_plus_1 = F * x_k; % Predicted state estimate
    P_k_plus_1 = F * P_k * F' + Q; % Predicted covariance estimate

    % Measurement update (PWE-estimated location)
    z_k = pwe_estimated_position';

    % Kalman gain calculation
    S_k = H * P_k_plus_1 * H' + R; % Innovation covariance
    K_k = P_k_plus_1 * H' / S_k; % Kalman gain

    % Update the state estimate
    x_k = x_k_plus_1 + K_k * (z_k - H * x_k_plus_1);

    % Update the error covariance
    P_k = (eye(4) - K_k * H) * P_k_plus_1;

    % Save the predicted position
    kalman_predicted_path(t, :) = x_k( 1 : 2 ); % Predicted position from Kalman filter

```

```

% Kalman Filter Implementation - Non-General Variance Estimation
% Predict the next state
x_k_plus_1_dynamic = F * x_k_dynamic;          % Predicted state estimate
P_k_plus_1_dynamic = F * P_k_dynamic * F' + Q;  % Predicted covariance estimate

% Initialize variance matrix
dynamic_variance_matrix = zeros( 2 , 2 );

% Compute weighted expectation
for kk = 1 : N_train

    yk = Y_train( kk, : ).'; % Training location
    dynamic_variance_matrix = dynamic_variance_matrix + weights_loc( kk ) * ( yk * yk.' + cov_ky );

end

% Store variance matrix
R_dynamic = dynamic_variance_matrix - pwe_estimated_position' * pwe_estimated_position;

% Kalman gain calculation with dynamic R
S_k_dynamic = H * P_k_plus_1_dynamic * H' + R_dynamic; % Innovation covariance
K_k_dynamic = P_k_plus_1_dynamic * H' / S_k_dynamic;    % Kalman gain

% Update the state estimate
x_k_dynamic = x_k_plus_1_dynamic + K_k_dynamic * ( z_k - H * x_k_plus_1_dynamic );

% Update the error covariance
P_k_dynamic = (eye(4) - K_k_dynamic * H) * P_k_plus_1_dynamic;

% Save the predicted position
dynamic_kalman_predicted_path( t , : ) = x_k_dynamic( 1 : 2 ); % Predicted position from dynamic
Kalman filter

% If the distance to the current stop point is less than the current movement increment,
% jump directly to the stop point. Once there, generate a new stop point and speed, then
% wait for the next time step.
if norm(stopPoint - true_position) <= speed * timeStep
    true_position = stopPoint;          % Jump to stop point
    stopPoint = grid_size * rand(1, 2); % Generate new stop point
    speed = 1 + 2 * rand;               % Generate new speed
else
    % The current stop point has not been reached so continue
    % travelling towards it at the current set speed.
    direction = (stopPoint - true_position) / norm(stopPoint - true_position);
    true_position = true_position + speed * timeStep * direction;
end
end

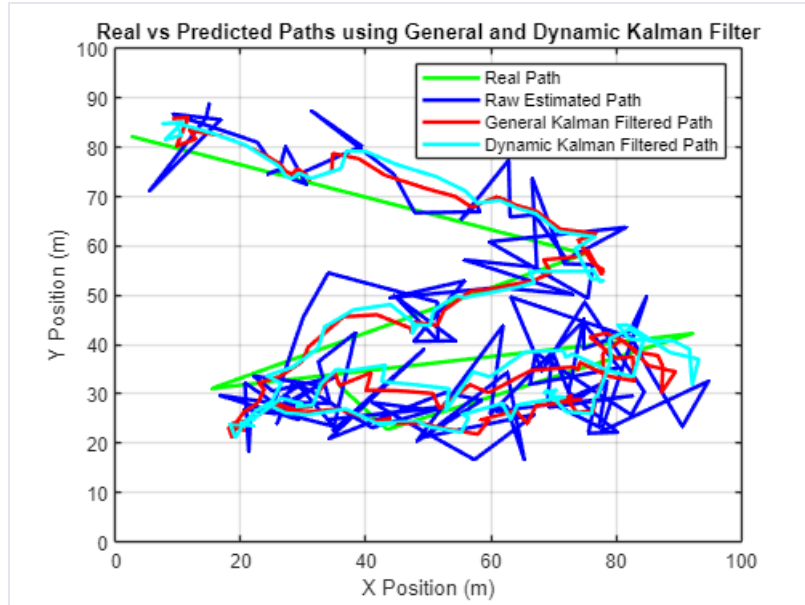
% Plot real and predicted paths for both implementations
figure;
plot(real_path(:, 1), real_path(:, 2), 'g-', ...
     pwe_estimated_path(:, 1), pwe_estimated_path(:, 2), 'b-', ...
     kalman_predicted_path(:, 1), kalman_predicted_path(:, 2), 'r-', ...
     dynamic_kalman_predicted_path(:, 1), dynamic_kalman_predicted_path(:, 2), 'c-', 'LineWidth', 2);
legend('Real Path', 'Raw Estimated Path', 'General Kalman Filtered Path', 'Dynamic Kalman Filtered Path');
title('Real vs Predicted Paths using General and Dynamic Kalman Filter');
xlabel('X Position (m)');

```

```

ylabel('Y Position (m)');
axis([0 grid_size 0 grid_size]);
grid on;
pwe_MSE = mean( sum( abs( real_path( : , 1:2 ) - pwe_estimated_path( : , 1 : 2 ) ).^2 , 2 ) )
kalman_MSE = mean( sum( abs( real_path( : , 1:2 ) - kalman_predicted_path( : , 1 : 2 ) ).^2 , 2 ) )
dynamic_kalman_MSE = mean( sum( abs( real_path( : , 1:2 ) - dynamic_kalman_predicted_path( : , 1 : 2 )
).^2 , 2 ) )

```



```
pwe_MSE = 110.5346
```

```
kalman_MSE = 49.0651
```

```
dynamic_kalman_MSE = 37.5206
```

Covariance of Process Noise - Tuning Parameter Optimization

```

% Define the objective function
objective_func = @(q) simulate_kalman(q, mean_variance_matrix, X_train, h_x, Y_train);

% Initial guess for Q elements
q_init = [0.4, 0.1, 0.5, 0.5];

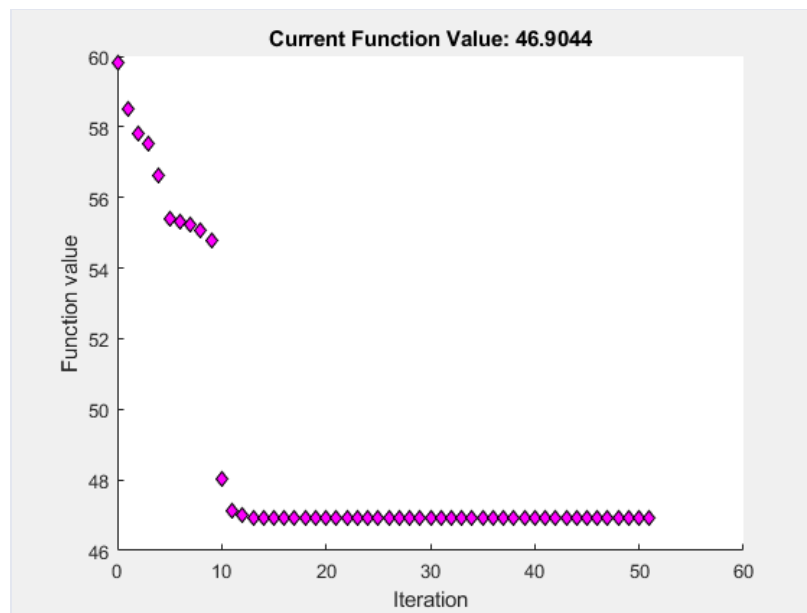
% Set optimization bounds for Q elements
lb = [0, 0, 0, 0]; % Lower bounds
ub = [10, 10, 10, 10]; % Upper bounds

% Minimize the objective function
options = optimset('Display', 'iter', 'PlotFcns', @optimplotfval);
optimized_q = fmincon(objective_func, q_init, [], [], [], [], lb, ub, [], options);
% Reshape optimized_q into matrix form
optimized_Q = diag(optimized_q)

```

Iter	F-count	f(x)	Feasibility	First-order optimality	Norm of step
0	5	5.981472e+01	0.000e+00	1.632e+00	
1	11	5.852449e+01	0.000e+00	2.747e+00	7.002e-01
2	17	5.780001e+01	0.000e+00	1.786e+00	2.059e-01
3	23	5.752228e+01	0.000e+00	6.044e+00	2.165e-01
4	28	5.661607e+01	0.000e+00	7.950e+01	2.493e+00
5	33	5.538838e+01	0.000e+00	1.284e+00	6.951e+00
6	38	5.532883e+01	0.000e+00	7.361e+00	2.049e+00
7	43	5.524089e+01	0.000e+00	2.697e+00	3.379e-01
8	48	5.506316e+01	0.000e+00	5.830e+01	1.758e+00
9	53	5.476083e+01	0.000e+00	2.106e+01	5.976e-01
10	58	4.800899e+01	0.000e+00	4.082e+02	2.687e+00
11	67	4.713381e+01	0.000e+00	1.147e+00	1.701e+00
12	72	4.701797e+01	0.000e+00	1.171e+01	6.906e-01
13	77	4.690819e+01	0.000e+00	2.905e+00	7.602e-01
14	82	4.691120e+01	0.000e+00	1.544e-01	2.245e-01
15	87	4.691454e+01	0.000e+00	5.005e-01	1.781e-01
16	92	4.691451e+01	0.000e+00	3.363e-01	3.958e-03
17	97	4.691448e+01	0.000e+00	2.526e-01	2.712e-03
18	102	4.691448e+01	0.000e+00	8.114e-02	3.576e-03
19	107	4.691448e+01	0.000e+00	2.000e-02	6.976e-04
20	112	4.690811e+01	0.000e+00	2.969e-02	2.763e-01
21	117	4.690661e+01	0.000e+00	1.007e-02	9.102e-02
22	122	4.690639e+01	0.000e+00	4.006e-03	1.847e-02
23	127	4.690517e+01	0.000e+00	2.829e-02	1.134e-01
24	132	4.690487e+01	0.000e+00	8.710e-04	4.226e-02
25	137	4.690455e+01	0.000e+00	6.823e-04	6.812e-02
26	142	4.690447e+01	0.000e+00	1.465e-03	3.415e-02
27	147	4.690445e+01	0.000e+00	1.628e-04	1.607e-02
28	152	4.690445e+01	0.000e+00	3.136e-05	6.092e-03
29	157	4.690445e+01	0.000e+00	8.068e-06	1.248e-03
30	162	4.690444e+01	0.000e+00	2.833e-05	5.377e-03
Iter	F-count	f(x)	Feasibility	First-order optimality	Norm of step
31	167	4.690444e+01	0.000e+00	1.031e-05	1.798e-03
32	172	4.690444e+01	0.000e+00	3.517e-06	2.582e-04
33	177	4.690444e+01	0.000e+00	4.590e-06	1.639e-05
34	184	4.690444e+01	0.000e+00	1.729e-06	8.058e-06
35	189	4.690444e+01	0.000e+00	2.152e-06	8.285e-07
36	196	4.690444e+01	0.000e+00	8.339e-06	5.736e-06
37	214	4.690444e+01	0.000e+00	1.682e-06	4.200e-07
38	226	4.690444e+01	0.000e+00	2.406e-03	2.123e-03
39	237	4.690444e+01	0.000e+00	2.642e-04	6.750e-04
40	243	4.690444e+01	0.000e+00	9.849e-04	3.491e-05
41	248	4.690444e+01	0.000e+00	5.016e-03	6.459e-04
42	253	4.690444e+01	0.000e+00	2.668e-03	3.038e-04
43	258	4.690444e+01	0.000e+00	3.808e-03	1.107e-03
44	263	4.690444e+01	0.000e+00	7.014e-03	9.628e-04
45	268	4.690444e+01	0.000e+00	5.233e-03	1.163e-03
46	273	4.690444e+01	0.000e+00	1.568e-03	5.226e-04
47	278	4.690444e+01	0.000e+00	4.506e-05	1.855e-04
48	283	4.690444e+01	0.000e+00	2.062e-06	1.181e-05
49	291	4.690444e+01	0.000e+00	5.008e-06	2.617e-07

50	296	4.690444e+01	0.000e+00	2.230e-06	8.154e-07
51	303	4.690444e+01	0.000e+00	7.475e-06	2.524e-06



Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than the value of the step size tolerance and constraints are satisfied to within the value of the constraint tolerance.

stopping criteria details>

```

optimized_Q =
    8.0495         0         0         0
         0    9.9962         0         0
         0         0    0.0243         0
         0         0         0    0.0085
  
```

Network Diagram

Training Results

Training finished: Met validation criterion ✔

Training Progress

Unit	Initial Value	Stopped Value	Target Value
Epoch	0	25	1000
Elapsed Time	-	00:00:09	-
Performance	6.52e+03	78.4	0
Gradient	1.36e+04	4.05	1e-07
Mu	0.001	0.1	1e+10
Validation Checks	0	6	6

Training Algorithms

Data Division: Random dividerand
 Training: Levenberg-Marquardt trainlm
 Performance: Mean Squared Error mse
 Calculations: MEX

Training Plots

Performance

Training State

Error Histogram

Regression

Final Proof of Concept - Dynamic Kalman Filter vs Neural Network Performance Comparison

```

% Construct a feedforward network with one hidden layer of size 10
net = feedforwardnet(10);

% Train the network
[net, tr] = train(net, X_train', Y_train');
% Initialize Random Waypoint Motion Model
timeStep = 1;                                % Poll position every second
simulationTime = 120;                          % Total simulation time
true_position = grid_size * rand(1, 2);        % Initial random starting position
stopPoint = grid_size * rand(1, 2);            % Initial random stop point

```

```

speed = 1 + 2 * rand;

% State-transition matrix (location and velocity)
F = [1 0 timeStep 0;
     0 1 0 timeStep;
     0 0 1 0;
     0 0 0 1];

% Observation matrix
H = [1 0 0 0; % Only position is measured, so H matrix maps state to measurement
     0 1 0 0];

% Covariance of process noise
Q = optimized_Q;

% Initial State Estimate (position and velocity)
x_k_dynamic = [true_position, 0, 0]';

% Initial error covariance
P_k_dynamic = [mean_variance_matrix, zeros(2, 2);
               zeros(2, 2), diag([10, 10])];

% Initialize matrices to store the real and predicted paths for plotting later
real_path = zeros( simulationTime , 2 );
dynamic_kalman_predicted_path = zeros( simulationTime , 2 );
nn_predicted_path = zeros( simulationTime , 2 );

% Main simulation loop
for t = 1 : timeStep : simulationTime

    % Save current real position
    real_path(t, :) = true_position;

    % Simulate RSS measurements
    rss_measurement = zeros(1, 4);
    for ii = 1 : 4
        distance_to_bs = norm( true_position - base_stations( ii , : ) ); % Distance to base station
        rss_measurement( ii ) = L0 - 10 * alpha * log10(4 * pi * distance_to_bs * f / c) + ( sigma_noise *
randn );
    end

    % Estimate position using PWE
    weights_loc = gaussian_kernel( rss_measurement , X_train , h_x );
    pwe_estimated_position = sum( weights_loc .* Y_train.' , 2 ).'; % PWE-estimated location

    % Dynamic Kalman Filter Localization
    % Predict the next state
    x_k_plus_1_dynamic = F * x_k_dynamic; % Predicted state estimate
    P_k_plus_1_dynamic = F * P_k_dynamic * F' + Q; % Predicted covariance estimate

    % Measurement update (PWE-estimated location)
    z_k = pwe_estimated_position';

    % Initialize variance matrix
    dynamic_variance_matrix = zeros( 2 , 2 );

    % Compute weighted expectation
    for kk = 1 : N_train

```

```

        yk = Y_train( kk, : ).'; % Training location
        dynamic_variance_matrix = dynamic_variance_matrix + weights_loc( kk ) * ( yk * yk.' + cov_ky );

    end

    % Covariance of measurement noise
    R_dynamic = dynamic_variance_matrix - pwe_estimated_position' * pwe_estimated_position;

    % Kalman gain calculation with dynamic R
    S_k_dynamic = H * P_k_plus_1_dynamic * H' + R_dynamic; % Innovation covariance
    K_k_dynamic = P_k_plus_1_dynamic * H' / S_k_dynamic; % Kalman gain

    % Update the state estimate
    x_k_dynamic = x_k_plus_1_dynamic + K_k_dynamic * ( z_k - H * x_k_plus_1_dynamic );

    % Update the error covariance
    P_k_dynamic = (eye(4) - K_k_dynamic * H) * P_k_plus_1_dynamic;

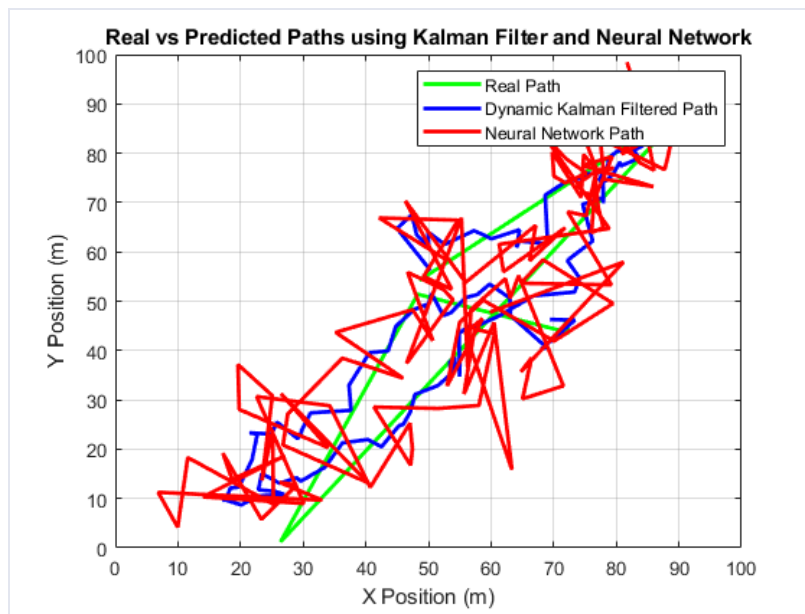
    % Save the predicted position
    dynamic_kalman_predicted_path( t , : ) = x_k_dynamic( 1 : 2 ); % Predicted position from dynamic
    Kalman filter

    % Neural Network Localization
    nn_predicted_position = net(rss_measurement'); % NN predicted position
    nn_predicted_path(t, :) = nn_predicted_position'; % Store NN predicted position

    % If the distance to the current stop point is less than the current movement increment,
    % jump directly to the stop point. Once there, generate a new stop point and speed, then
    % wait for the next time step.
    if norm(stopPoint - true_position) <= speed * timeStep
        true_position = stopPoint; % Jump to stop point
        stopPoint = grid_size * rand(1, 2); % Generate new stop point
        speed = 1 + 2 * rand; % Generate new speed
    else
        % The current stop point has not been reached so continue
        % travelling towards it at the current set speed.
        direction = (stopPoint - true_position) / norm(stopPoint - true_position);
        true_position = true_position + speed * timeStep * direction;
    end
end

% Plot real and predicted paths for both implementations
figure;
plot(real_path(:, 1), real_path(:, 2), 'g-', ...
     dynamic_kalman_predicted_path(:, 1), dynamic_kalman_predicted_path(:, 2), 'b-', ...
     nn_predicted_path(:, 1), nn_predicted_path(:, 2), 'r-', 'LineWidth', 2);
legend('Real Path', 'Dynamic Kalman Filtered Path', 'Neural Network Path');
title('Real vs Predicted Paths using Kalman Filter and Neural Network');
xlabel('X Position (m)');
ylabel('Y Position (m)');
axis([0 grid_size 0 grid_size]);
grid on;
dynamic_kalman_MSE = mean( sum( abs( real_path( : , 1:2 ) - dynamic_kalman_predicted_path( : , 1 : 2 )
).^2 , 2 ) )
nn_MSE = mean( sum( abs( real_path( : , 1:2 ) - nn_predicted_path( : , 1 : 2 ) ).^2 , 2 ) )

```



dynamic_kalman_MSE = 50.6617

nn_MSE = 114.7496

A. gaussian_kernel.m

```
function weights = gaussian_kernel(X_i, X_train, h)

[ N_i , M_i ] = size( X_i );
[ N_train , M_train ] = size( X_train );
weights = zeros( N_i , N_train );

% Run through all signals to estimate the weights
for nn = 1 : N_i

    % Find sum of squared distance from observation to all training
    % observations normalized by kernel width.
    delta = sum( abs( ( X_i( nn , : ) - X_train ) / h ).^2 , 2 );

    % Compute exponential of sum.
    weights( nn , : ) = exp( -0.5 * delta );

    % Normalize weights for sum of unity.
    weights( nn , : ) = weights( nn , : ) / sum( weights( nn , : ) );

end
```

B. simulate_kalman.m

```
function mse = simulate_kalman( q , mean_variance_matrix , X_train , h_x , Y_train )

rng(42);

% Simulation environment parameters
grid_size = 100;           % Size of grid (100m x 100m)
sigma_noise = 2;           % Noise variance level
base_stations = [25, 25;   % Base station location 1
                 25, 75;   % Base station location 2
                 75, 25;   % Base station location 3
                 75, 75]; % Base station location 4

% Path loss model parameters
L0 = -30; % RSS at reference distance of 0m (perfect signal connection), in dBm
alpha = 2; % Path loss exponent
f = 2.4e9; % Frequency in Hz (2.4 GHz for WLAN/WiFi)
c = 3e8; % Speed of light in m/s

% Initialize Random Waypoint Motion Model
timeStep = 1; % Poll position every second
simulationTime = 120; % Total simulation time
currentPosition = grid_size * rand(1, 2); % Initial random starting position
stopPoint = grid_size * rand(1, 2); % Initial random stop point
speed = 1 + 2 * rand;

% State-transition matrix (location and velocity)
F = [1 0 timeStep 0;
```

```

    0 1 0 timeStep;
    0 0 1 0;
    0 0 0 1];

% Observation matrix
H = [1 0 0 0; % Only position is measured, so H matrix maps state to measurement
     0 1 0 0];

% Covariance of process noise
Q = diag(q);

% Covariance of measurement noise
R = mean_variance_matrix; % This value is the mean variance estimation calculated previously in
evaluation process for PWE

% Initial State Estimate (position and velocity)
x_k = [currentPosition, 0, 0]'; % General Variance Estimation

% Initial error covariance
P_k = [mean_variance_matrix, zeros(2, 2);
       zeros(2, 2), diag([10, 10])]; % General Variance Estimation

% Initialize matrices to store the real and predicted paths for plotting later
real_path = zeros( simulationTime , 2 );
pwe_estimated_path = zeros( simulationTime , 2 );
kalman_predicted_path = zeros( simulationTime , 2 );

% Main simulation loop
for t = 1 : timeStep : simulationTime

    % Save current real position
    real_path(t, :) = currentPosition;

    % Simulate RSS measurements
    rss_measurement = zeros(1, 4);
    for ii = 1 : 4
        distance_to_bs = norm( currentPosition - base_stations( ii , : ) ); % Distance to base station
        rss_measurement( ii ) = L0 - 10 * alpha * log10(4 * pi * distance_to_bs * f / c) + (
sigma_noise * randn );
    end

    % Estimate position using PWE
    weights_loc = gaussian_kernel( rss_measurement , X_train , h_x );
    estimated_position = sum( weights_loc .* Y_train.' , 2 ).'; % PWE-estimated location

    % Save the PWE estimated position
    pwe_estimated_path( t , : ) = estimated_position;

    % Kalman Filter Implementation - General Variance Estimation
    % Predict the next state
    x_k_plus_1 = F * x_k; % Predicted state estimate
    P_k_plus_1 = F * P_k * F' + Q; % Predicted covariance estimate

    % Measurement update (PWE-estimated location)
    z_k = estimated_position';

    % Kalman gain calculation
    S_k = H * P_k_plus_1 * H' + R; % Innovation covariance

```

```

K_k = P_k_plus_1 * H' / S_k;    % Kalman gain

% Update the state estimate
x_k = x_k_plus_1 + K_k * (z_k - H * x_k_plus_1);

% Update the error covariance
P_k = (eye(4) - K_k * H) * P_k_plus_1;

% Save the predicted position
kalman_predicted_path(t, :) = x_k( 1 : 2 ); % Predicted position from Kalman filter


% If the distance to the current stop point is less than the current movement increment,
% jump directly to the stop point. Once there, generate a new stop point and speed, then
% wait for the next time step.
if norm(stopPoint - currentPosition) <= speed * timeStep
    currentPosition = stopPoint;          % Jump to stop point
    stopPoint = grid_size * rand(1, 2); % Generate new stop point
    speed = 1 + 2 * rand;                 % Generate new speed
else
    % The current stop point has not been reached so continue
    % travelling towards it at the current set speed.
    direction = (stopPoint - currentPosition) / norm(stopPoint - currentPosition);
    currentPosition = currentPosition + speed * timeStep * direction;
end
end

% Return MSE as the objective value
mse = mean( sum( abs( real_path( : , 1:2 ) - kalman_predicted_path( : , 1 : 2 ) ).^2 , 2 ) );

end

```

References

1. D. Felix and M. McGuire, "Received signal strength calibration for handset localization in WLAN," *2012 IEEE International Conference on Communications (ICC)*, Ottawa, ON, Canada, 2012, pp. 3664–3669, doi: 10.1109/ICC.2012.6364358.
2. "Wi-Fi and Bluetooth contact tracing without user intervention," *IEEE Xplore*. [Online]. Available: <https://ieeexplore.ieee.org/document/9866766/> [Accessed: Oct. 9, 2024].
3. M. McGuire, K. N. Plataniotis, and A. N. Venetsanopoulos, "Estimating position of mobile terminals from path loss measurements with survey data," *Wireless Communications and Mobile Computing*, vol. 3, no. 1, pp. 51–62, 2003, doi: 10.1002/wcm.49.
4. "Free-space path loss," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Free-space_path_loss [Accessed: Sep. 30, 2024].
5. D. Felix, "Received Signal Strength Calibration for Wireless Local Area Network Localization," M.A.Sc. thesis, Dept. Elect. and Comput. Eng., Univ. Victoria, Victoria, BC, Canada, 2010.
6. D. Felix, E. Hyun, M. McGuire, and M. Sima, "Denoising by spatial domain averaging for wireless local area network terminal localization," *ResearchGate*, 2010. [Online]. Available: <https://www.researchgate.net/publication/289794102> [Accessed: Sep. 30, 2024].

7. J. Schneider, "Cross Validation," *cmu.edu*. [Online]. Available: <https://www.cs.cmu.edu/~schneide/tut5/node42.html>
[Accessed: Sep. 30, 2024].